The Two *Calendars*

Solution Document — Calendar alignment in marketing mix modelling

1 Solution overview

Weekly TV media spend was being combined with calendar-month digital/email spend and sales, creating a potential timing error in an already implemented 15% TV reallocation. I built a reproducible synthetic experiment, two calendar-conversion paths, a constrained monthly marketing mix model with adstock and saturation, and a refresh gate with independently verified LLM specification review. In the primary three-month equal digital/email transfer scenario, correcting the calendar changes estimated sales lift from 1.31% to 1.11%; the corrected model-based 95% interval is 0.66% to 1.81%. Both versions support this defined action within the synthetic model, while broad attribution uncertainty and unresolved vendor details prevent treating the result as proof about the real brand.

PRIMARY RESULT · CORRECTED MODEL

+28.00

simulated revenue units of incremental sales

+16.74 to +45.44

95% interval, paired residual block bootstrap

Scenario: TV reduced to 85% of the latest corrected quarter’s budget; the removed 15% transferred 50/50 to digital and email (assumed split); three-month horizon; total spend unchanged. Synthetic data and a model-based interval — not a verified result for the real brand.

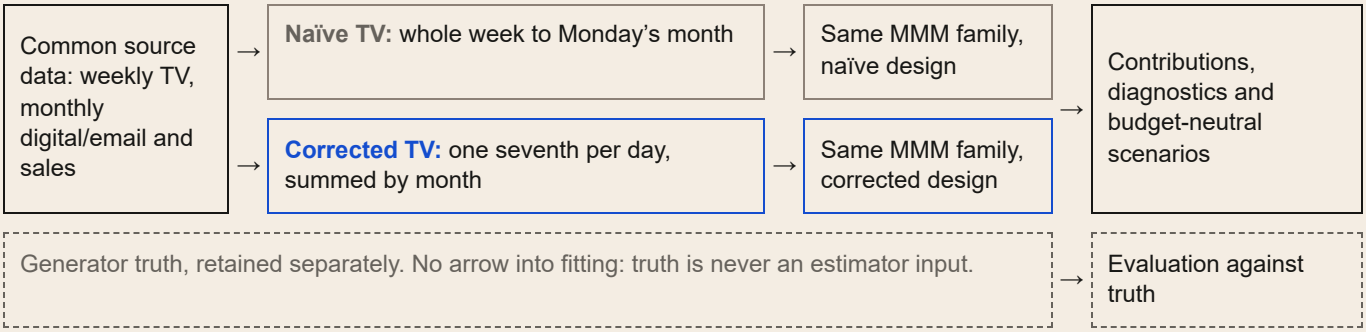
Case context

The case reports prior-quarter sales contributions of TV 22%, digital 19% and email 11%; the remaining 48% is unspecified. These historical figures are neither generated truth nor fitting targets. The Q3 presentation deadline of “in 10 days” belongs to the case narrative, not to a newly calculated real-world date.

2 Approach and methodology

1. **Generate vendor-shaped synthetic inputs and retain known effects separately.** Makes calendar-related estimation error measurable without claiming access to real client data.
2. **Validate before estimation.** Schemas, grains, dates, frequency, coverage, joins, numeric values and declared synthetic units are checked so faulty inputs never reach the model.
3. **Run two TV paths.** The faulty path assigns each weekly amount to its Monday's month; the corrected path allocates one seventh to each date, then aggregates by calendar month. Digital/email and sales stay monthly.
4. **Fit the same model family independently on both designs,** with the same seeded candidate-transform set and fitting protocol. Only the TV calendar treatment changes.
5. **Evaluate a final-12-month chronological holdout** trained on earlier data; then refit on all 60 analysis months for final attribution and scenarios.
6. **Use a paired residual block bootstrap** to compare contributions and to simulate the actual budget action, rather than average sales/spend ratios.
7. **Save the evidence:** offline HTML report, tests, validation status, review provenance and reproduction results.

Two calendar paths, one model family



2 Approach and methodology continued

Worked example: one week across two months

June		July				
Mon 29	Tue 30	Wed 01	Thu 02	Fri 03	Sat 04	Sun 05
10,000	10,000	10,000	10,000	10,000	10,000	10,000
Naïve: June 70,000		July 0				
Corrected: June 20,000		**July 50,000 · total 70,000 conserved**				

Figure 2. Monday 29 June–Sunday 5 July 2026, raw currency units (generated spend tables use simulated currency thousands). Equal daily delivery is an explicit assumption; weekly totals do not establish actual day-level delivery.

Model

```
adstock[c,t]  = spend[c,t] + decay[c] × adstock[c,t−1]
response[c,t] = adstock[c,t] / (half[c] + adstock[c,t])
sales[t]      = intercept + trend + annual sine + annual cosine
                + Σ beta[c] × response[c,t] + error[t]
beta[c] ≥ 0 for media coefficients; controls unconstrained
```

The estimator searches 180 reproducible joint transform candidates (candidate RNG seed 7301). Decay is bounded to 0.05–0.80; half-saturation to 0.35–4.0 times the pre-holdout mean channel spend, including warm-up. For each candidate it enumerates media active sets, solves constrained least squares and keeps the minimum training SSE. This is a finite candidate search, not a proven continuous global optimum. Generator parameters are never used for fitting.

Windows

Warm-up is January–December 2020: its sales are excluded from fitting, but its spend initializes adstock. Training covers 2021–2024 and the holdout is January–December 2025. Final reporting refits on 2021–2025.

2 Approach and methodology continued

Uncertainty

200 paired circular residual block-bootstrap draws with three-month blocks. Residuals are centred, chronological designs are retained, and transform selection and coefficient fitting are repeated inside every draw, using the same index draws for both calendar versions. Contribution denominators stay at original observed sales. Final intervals therefore include reselection within the finite nonlinear search. They remain approximate and do not cover every causal or specification uncertainty.

Budget scenario

The latest corrected quarter’s budgets are repeated over a hypothetical three-month horizon. TV is reduced to 85% and exactly the removed amount is transferred: 50/50 digital/email in the primary case, with 0/100, 25/75, 75/25 and 100/0 as sensitivities. Both models receive the same physical schedule; baseline and intervention share historical adstock state within each model. Carryover inside the horizon is included; no post-horizon tail is modelled. Neither 15% nor the 50/50 split is claimed to be optimal.

Sources: `src/core.mjs`, `src/model.mjs`, `src/refresh.mjs`, `run_pipeline.mjs`, `outputs/model_results.json`

3 How you arrived at it

Alternatives considered

Decision	Reason and status
Custom synthetic generator rather than a public sample	Supplies explicit known channel effects and exact source-calendar control. Public samples were not downloaded or benchmarked.
Lightweight custom MMM rather than Robyn, Meridian or a large Bayesian platform	Inspectable implementation with no package installation. No performance superiority over those platforms is claimed.
Monthly model rather than fabricated weekly outcomes	Sales and digital/email have an actual monthly grain.
Day-overlap allocation rather than treating adstock as a calendar fix	Adstock models carryover; it cannot correct mislabelled source timing.
Node.js rather than the suggested Python structure	The `python` command was unavailable in the execution PATH; Node.js was available.
Refit transforms in each bootstrap draw rather than fixing them	Broadens uncertainty to include candidate-transform selection.
Live API review plus bound replay rather than replay alone	Demonstrates genuine runtime LLM execution while keeping offline reproduction key-free.

Key assumptions

Equal daily allocation of each TV week	Unspecified transfer split, represented by a primary 50/50 assumption
Generator and estimator share the same model family	Three-month scenario horizon chosen for illustration
Controlled synthetic units throughout	No commercial materiality threshold was supplied

Clarifications and interpretation

No external Clarification Desk answer is relied upon in this implementation record. The case’s weekly-TV and monthly-sales definitions determine the alignment problem; equal daily delivery and the digital/email transfer split are explicit project assumptions rather than attributed desk answers. The vendor’s actual currency/scale, timezone, fiscal anchor, label semantics and within-week delivery remain information to confirm before real-account use.

4 Data

Dataset	Rows / coverage	Role
Weekly TV	314 complete weeks; 30 Dec 2019–4 Jan 2026	Vendor-shaped source spanning month boundaries
Monthly digital/email	72 rows; Jan 2020–Dec 2025	Monthly spend inputs
Monthly sales	72 rows; Jan 2020–Dec 2025	Monthly outcome
Analysis window	60 months; Jan 2021–Dec 2025	Estimation and reporting; final 12 months held out for diagnostics
Warm-up	12 months; Jan–Dec 2020	Initializes adstock

Generation recipe and built-in truth

Seed 42 was fixed before the main results. TV contains random weekly variation and independently placed bursts; digital/email vary monthly. Sales add known channel responses to a baseline with modest trend, annual seasonality and mean-zero noise. Spend is in simulated currency thousands; sales are simulated revenue units. No public dataset and no actual patient or client data were used.

Channel	True decay	True half-saturation	True effect coefficient
TV	0.45	550	280
Digital	0.25	250	210
Email	0.15	90	110

Baseline generator settings: intercept 430; trend 1.1 revenue units per month; annual sine amplitude 28; cosine amplitude 18; noise SD 14. These are generator settings, not estimated coefficients. Same-family simulation deliberately favours recovery relative to real-world MMM. True contribution is the summed noiseless channel effect divided by the same observed-sales total used for fitted shares; the three media shares need not sum to 100%.

Conservation evidence, simulated currency thousands

25,466.34
Full source weeks = full day allocations

20,651.23
Corrected analysis window, matching explicit source-day overlap

20,649.87
Naïve analysis window; boundary difference −1.36

Mean absolute monthly timing adjustment is 43.00. Full-calendar conservation does not require equal totals in a cropped window, so the data are not rescaled to hide boundary effects.

Sources: raw and processed CSVs, `outputs/config.json`, `outputs/truth.json`, `outputs/calendar_differences.csv`

5 Evaluation

Channel contributions, % of observed sales, 2021–2025

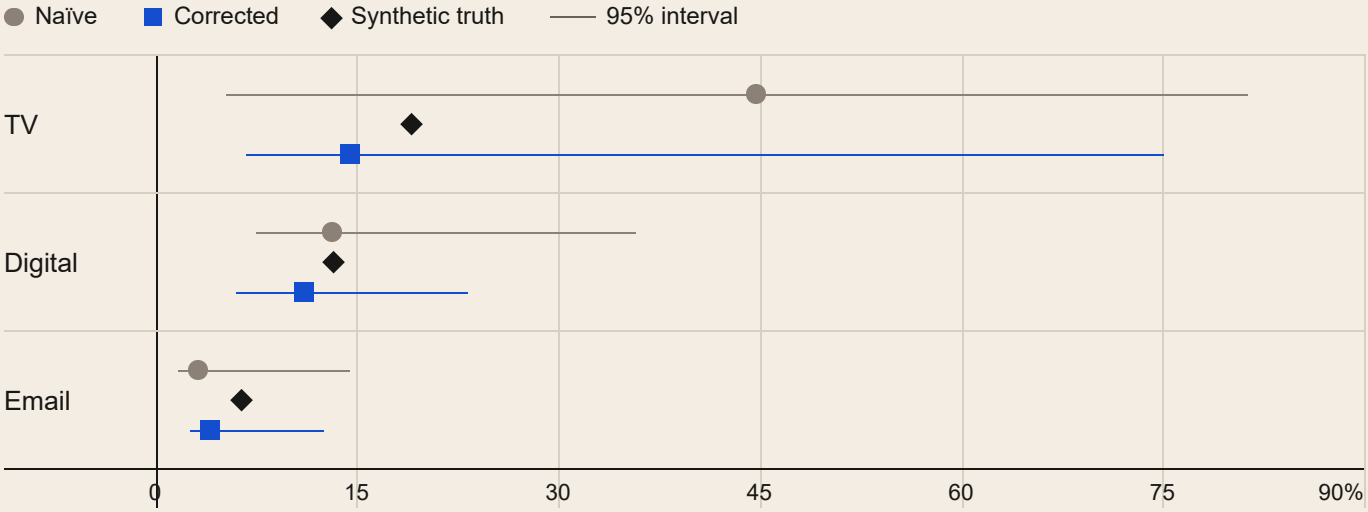


Figure 5.1. Share of observed sales, January 2021–December 2025, common denominator for all series. Intervals: 95% paired circular residual block bootstrap (200 draws, three-month blocks) with transform reselection in each draw.

Channel	Naïve [95% interval]	Corrected [95% interval]	Synthetic truth
TV	44.71 [5.29, 81.33]	14.51 [6.79, 75.07]	19.05
Digital	13.15 [7.48, 35.85]	11.11 [6.02, 23.28]	13.23
Email	3.21 [1.73, 14.55]	4.04 [2.58, 12.56]	6.42

Paired change, corrected minus naïve

TV −30.20 pp [−61.77, 31.69]; digital −2.04 pp [−23.17, 8.40]; email +0.82 pp [−3.44, 5.79]. All three paired intervals cross zero (pp = percentage points). The TV point estimate moves substantially, but the evidence does not establish a precisely estimated attribution change.

5 Evaluation continued

Chronological holdout, 2025, sales units (lower is better)

Metric	Naïve	Corrected
RMSE	11.79	15.65
MAE	9.27	12.33
Controls-only RMSE	28.43	28.43

The naïve model predicts this holdout better. Calendar correction is still required for valid timing: predictive fit alone does not establish causal attribution.

Primary 15% shift: three months, equal digital/email split

Version	Revenue lift [95% interval]	Relative lift [95% interval]
Naïve	+32.95 [19.75, 52.35]	+1.31% [0.78%, 2.07%]
Corrected	+28.00 [16.74, 45.44]	+1.11% [0.66%, 1.81%]
Generator benchmark	+31.75	+1.27%

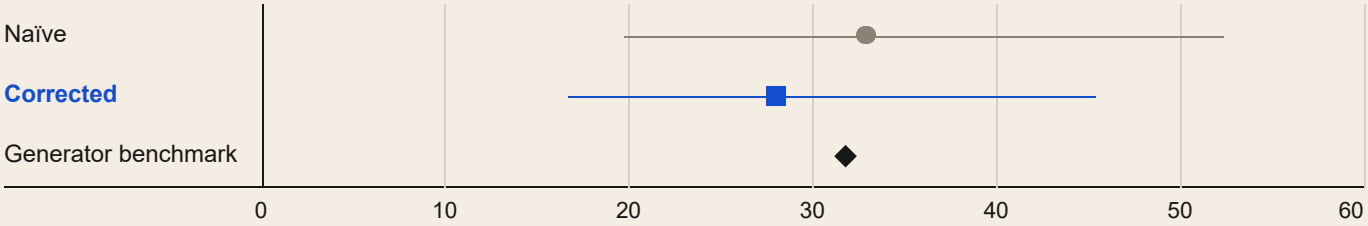


Figure 5.2. Incremental sales over the three-month horizon, simulated revenue units, zero reference at left. TV at 85% of the latest corrected quarter; 15% moved 50/50 to digital and email; same schedule for both models. The noiseless generator benchmark has no interval.

Both fitted primary intervals are above zero, so both models support the defined action under the model. This is not a verified real-brand recommendation. Primary bootstrap support is 200/200 positive draws, which is not a calibrated 100% probability of success.

Robustness seeds

Seeds 17, 83 and 101 were retained without replacing seed 42 by a more favourable one. Mean absolute contribution error, naïve → corrected: 3.10 → 3.19 pp, 2.71 → 3.30 pp and 30.23 → 5.48 pp. Correction improves this metric in one case and worsens it in two. One naïve seed-101 fit attributes over 100% to TV with offsetting controls, which indicates weak identification rather than a credible economic decomposition.

5 Evaluation continued

Corrected split sensitivity

Digital / email share of transfer	Revenue lift [95% interval]	Positive bootstrap draws
0 / 100	+24.59 [10.56, 56.36]	100.0%
25 / 75	+27.32 [14.90, 52.70]	100.0%
50 / 50 (primary)	+28.00 [16.74, 45.44]	100.0%
75 / 25	+26.28 [15.71, 37.88]	100.0%
100 / 0	+21.64 [11.40, 31.47]	99.5%

All five model-based intervals are above zero (simulated revenue units, three-month horizon). Across the top 20 transform candidates the primary lift ranges from 27.44 to 33.43 units; this is a descriptive sensitivity envelope, not a confidence interval.

Tests and verification

65 automated tests passed and none failed; 11 source checks and 12 artifact checks passed; each model has 200 valid bootstrap draws with zero failures. A clean run using the saved genuine API review reproduced all numerical results exactly, with no key and no network call.

Tests cover month and year boundaries, leap day, zero spend, conservation, invalid grains/dates/joins/units, constrained fitting, scenario budgets, bootstrap blocks, stale reviews, provider errors/refusals/timeouts, evidence references, credential non-disclosure and ZIP integrity. Test mocks are not provider execution evidence; the accepted live bundle is kept separately.

Updates during implementation

The available record does not identify a separate assessor-issued mid-session amendment. The recorded implementation revisions were to refit the finite adstock/saturation search within each bootstrap draw, upgrade the specification review from Codex-context replay to a real OpenAI API adapter, and replace model-copied quotations with numbered evidence-line selection after an exact-quote check rejected the first live attempt. These changes strengthened uncertainty coverage and review auditability; the final numerical findings and accepted live evidence are reported above. The first rejected request's raw response was not retained by that earlier adapter version.

Sources: `outputs/model_results.json`, `outputs/scenarios.csv`, `outputs/bootstrap_draws.json`, `outputs/robustness.json`, `outputs/test_results.tap`, `outputs/reproducibility.json`, `PROGRESS.md`

6 Tech stack

Layer	Actual implementation
Language / runtime	JavaScript ES modules; Node.js v21.7.3 tested on Windows; declared minimum Node 18
Numerical work	Custom seeded RNG, date allocation, constrained least squares, transform search and bootstrap; no external numerical packages
Built-in modules	Node filesystem/path, crypto, zlib, child process, OS, test/assert, native fetch/AbortSignal
API	Direct HTTPS OpenAI Responses API with strict JSON Schema; no OpenAI SDK dependency
Data / artifacts	Local CSV/JSON; self-contained HTML with embedded SVG; Markdown; portable ZIP with SHA-256
Developer / verification tools	PowerShell, VS Code, Chrome via computer-use tooling; independent .NET ZIP extraction and hash check
Solution Document	A4 HTML/CSS page layout (Arial, Georgia Italic, Consolas), exported with browser print-to-PDF; the exporting browser version is not recorded in this document
Infrastructure	Local file-based study; no deployed app, database or hosted dashboard; optional external OpenAI API request

Every LLM used

Role	Provider, model and version
Implementation assistance and original specification review	OpenAI Codex assistant. The exact serving model and version were not exposed in the retained development record; the originally saved GPT-6-family label is not an exact audited backend version.
Live specification reviewer (runtime)	OpenAI; requested alias `gpt-5.4-mini`; served model `gpt-5.4-mini-2026-03-17`, retained in the provider response
Solution Document layout and design assistance	Anthropic Claude, used through a hosted design tool. The exact model version was not exposed in the authoring session.

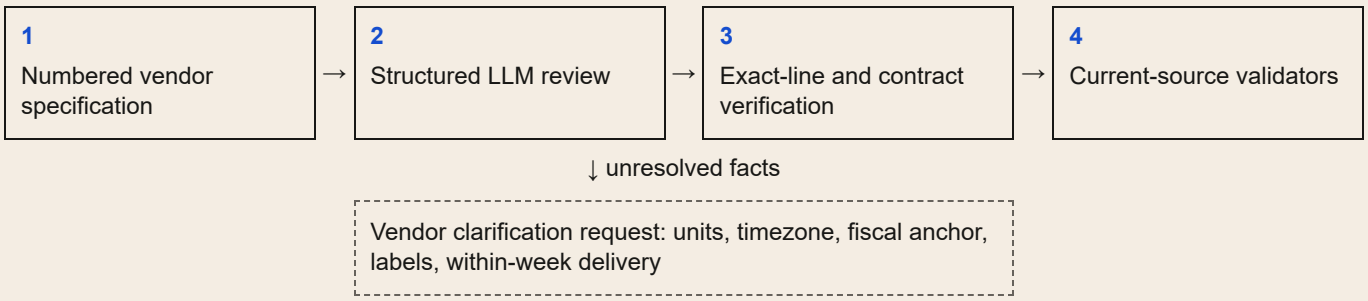
Not used: Python, NumPy, pandas, SciPy, pytest, Robyn, Meridian, PyMC, Stan or any frontend framework.

7 Use of LLMs

Where an LLM is used, and why

The LLM interprets vendor prose: it identifies which calendar a source uses, whether it lines up with calendar months, and which assumptions are ambiguous. That reading is semantic and varies with wording, which fixed rules handle poorly. Deterministic code owns arithmetic, date handling, model fitting, truth evaluation and every pass/fail check. The LLM is not the MMM, does not determine true channel effects and does not replace causal identification.

Review pipeline



Grounding and controls

- The vendor specification is treated as untrusted data, never as executable instructions.
- The model selects supporting line IDs; code extracts the exact quotations and verifies source and contract compatibility.
- Each review is bound to the exact specification bytes and SHA-256. A changed specification requires a new genuine review; stale live evidence never silently falls back to an old response.
- Retained: prompt, key-free request body, raw and parsed response, served model, response/request IDs, timestamps and usage. Replay cross-checks provider metadata and raw content against the accepted review.
- Independent current-data checks run on every refresh; a linked check passing does not prove every semantic assertion in a flag.
- Missing units, fiscal, timezone and delivery facts stay a vendor information request, never a verified assumption.

7 Use of LLMs continued

Actual execution

The accepted live request ran on 3 November 2026 at 13:28:29 IST; the served model was `gpt-5.4-mini-2026-03-17`. The final saved verification operation is a replay of that genuine API response, not a second API request. Full identifiers are in `outputs/llm_live_review.json`.

Live flag	Independent evidence and disposition
Weekly TV uses Monday–Sunday media-buying weeks	Date-boundary validator passed. Dates alone do not establish an unprovided vendor fiscal anchor or 4-4-5 grouping.
Media calendar does not line up with calendar months	Linked corrected monthly-join coverage passed; day allocation, boundary fixtures and conservation tests add implementation evidence.
Currency, scale, timezone, fiscal anchor, labels and within-week delivery unspecified	Grouped into one unresolved vendor clarification. Passing synthetic schema checks does not resolve these real-source facts.

Known failure modes

Paraphrased or hallucinated evidence, invalid line IDs, unsupported source calendars, missing or duplicate source descriptions, malformed schema or JSON, refusal, incomplete output, timeout, HTTP 401/403/429/500, stale reviews, and treating assumptions as verified facts. The first live attempt demonstrated exact-quote rejection; numbered-line extraction corrected it without weakening validation.

Small deterministic fixture tests and one accepted vendor-spec execution are not a broad LLM accuracy benchmark.

Security settings

The API key is supplied only through the caller’s environment and never appears in audit bodies or the submission. The adapter uses a fixed HTTPS endpoint, rejects redirects, exposes no tools, sets `store:false` and makes no automatic retries. Limits: 48,000-byte input, 6,000-token output, 60-second timeout. These settings do not guarantee zero provider retention or full production security.

8 Limitations and next steps

Limitation	Change for a real client deployment
Synthetic, same-family simulation	Reconcile actual account data; assess alternative response families and confounding before any real recommendation
Unknown within-week delivery and fiscal semantics	Obtain actual delivery dates, a calendar/label dictionary, the fiscal anchor, and confirmed units and timezone
Weak attribution and a broad TV interval	Improve identification; examine constraints, controls and richer independent variation; do not present precise channel shares
Finite transform grid and model-based bootstrap	Expand specification and block-length sensitivity; use uncertainty methods appropriate to the real data
Three-month illustrative scenario	Confirm the actual transfer split, planning budgets, commercial thresholds and carryover horizon
Local prototype	Add durable orchestration, versioned data contracts, controlled storage and operational ownership
Privacy and security not production-certified	Minimize and de-identify data; keep patient-level information out of prompts; review provider terms, retention and approved data flows; use managed secrets, least privilege and access logs
No ongoing production monitoring	Monitor missing/duplicate periods, grain and calendar drift, unit changes, spend reconciliation, spec-hash changes, API failures, residuals, holdout performance and attribution stability; alert and block publication on critical failures

Recommended stakeholder actions

1. Notify the account lead and Brand Director promptly of the calendar discrepancy and the bounded investigation.
2. Qualify any Q3 claims the evidence does not support.
3. Reconcile vendor facts and actual account data.
4. Review the corrected evidence before the case’s presentation deadline.

These are recommended actions, not messages already sent. A passing synthetic gate is not production sign-off.

9 Submission index

Paths are relative to the project root. “Imported” marks a module with no standalone command; “Read” marks an evidence artifact rather than executable code. Command letters refer to the reproduction list at the end of this section.

File	What it is	How to run or consume
`README.md`	Setup, findings, assumptions and commands	Read
`PROGRESS.md`	Implementation and verification history	Read
`.gitignore`	Excludes local secrets and dependencies	Used by Git
`package.json`	Runtime metadata and npm command aliases	Read / npm scripts
`config.json`	Seed, windows and analysis settings	Read by pipeline
`run_pipeline.mjs`	Complete generator, model and report driver	Command A
`review_spec.mjs`	Live/replay LLM review and matching report refresh	Commands D, E
`validate_refresh.mjs`	Source and review verification gate	Command C
`robustness.mjs`	Separate fixed-seed sensitivity runs	Command F
`verify_artifacts.mjs`	Numerical, link and provenance artifact checks	Command G
`reproduce.mjs`	Isolated clean numerical reproduction	Command H
`package_submission.mjs`	Portable verified ZIP and checksum writer	Command I
`package_submission.ps1`	Legacy wrapper for Node packaging	Optional; Node preferred
`src/core.mjs`	Generator, calendar allocation and validation helpers	Imported
`src/io.mjs`	Strict CSV reading and one-to-one source joins	Imported
`src/llm.mjs`	API adapter, schema, evidence extraction and replay	Imported
`src/model.mjs`	MMM design, solver, search, forecasts and bootstrap	Imported

9 Submission index continued

File	What it is	How to run or consume
src/refresh.mjs	Current-source checks and review binding	Imported
src/report.mjs	HTML and submission-memo generation	Imported
src/spec_review.mjs	Compatibility export for the original module layout	Imported if needed
src/zip.mjs	ZIP32/DEFLATE and CRC integrity implementation	Imported
specs/vendor_spec.txt	Vendor specification input	Read by review
specs/llm_review_original.json	Genuine original Codex-context response	Offline fallback evidence
specs/llm_review_original.manifest.json	Pinned original-spec hash	Read by replay
tests/calendar.test.mjs	Boundary, leap-year and conservation tests	Command B
tests/evidence.test.mjs	Provenance, bootstrap and input-leakage checks	Command B
tests/llm.test.mjs	Simulated adapter success and failure tests	Command B; no paid calls
tests/model.test.mjs	Solver, response and scenario tests	Command B
tests/validation.test.mjs	Invalid source, schema and join fixtures	Command B
tests/zip.test.mjs	CRC, path and ZIP integrity tests	Command B
data/raw/tv_weekly.csv	Generated weekly TV input	Read by gate and model
data/raw/digital_email_monthly.csv	Generated monthly media inputs	Read by gate and model
data/raw/sales_monthly.csv	Generated monthly sales input	Read by gate and model
data/processed/naive_monthly.csv	Faulty calendar design	Read / compare
data/processed/corrected_monthly.csv	Day-apportioned calendar design	Read / compare
outputs/SUBMISSION.md	Concise stakeholder and assessment memo	Read
outputs/artifact_verification.json	Latest artifact checks	Read

9 Submission index continued

File	What it is	How to run or consume
`outputs/bootstrap_draws.json`	Saved draws, scenarios and transform selections	Read / audit
`outputs/calendar_differences.csv`	Monthly naïve/corrected timing differences	Read / plot
`outputs/config.json`	Executed analysis configuration	Read
`outputs/llm_live_review.json`	Genuine accepted OpenAI API bundle	Read / replay; no key
`outputs/llm_review.json`	Current verified review and flag checks	Read
`outputs/llm_review_attempt.json`	Latest review-attempt status	Read
`outputs/model_results.json`	Contributions, diagnostics and scenarios	Primary numerical source
`outputs/refresh_validation.json`	Current disk-refresh result	Read
`outputs/report.html`	Offline before/after analytical report	Open in a browser
`outputs/reproducibility.json`	Clean-run numerical hash comparison	Read
`outputs/robustness.json`	Fixed-seed sensitivity results	Read
`outputs/run_manifest.json`	Actual runtime, command and source fingerprint	Read
`outputs/scenarios.csv`	All fitted reallocation scenarios	Read / plot
`outputs/test_results.tap`	Actual 65-test execution evidence	Read
`outputs/truth.json`	Generator settings and monthly true effects	Read; never fed to estimator
`outputs/validation.json`	Main pipeline’s source validation	Read
`outputs/visual_review.json`	Bounded Chrome inspection evidence and hashes	Read; not PDF QA
`outputs/SOLUTION_DOCUMENT.pdf`	Required nine-section Solution Document (this file)	Open PDF
`outputs/SOLUTION_DOCUMENT_source/`	Editable HTML source: SOLUTION_DOCUMENT.dc.html with doc-page.js and support.js	Open the .dc.html in a browser

Rows in blue were added with this document. The verified archive before they were added contained the 54 files above them; the final inventory is the rebuilt archive’s own listing.

9 Submission index continued

Reproduction commands

Run from the extracted project root with Node.js 18 or later. No dependency installation is required.

A	`node run_pipeline.mjs --seed 42 --bootstrap 200`
B	`node --test tests/*.test.mjs`
C	`node validate_refresh.mjs data/raw`
D	`node review_spec.mjs --replay`
E	`node review_spec.mjs --live`
F	`node robustness.mjs`
G	`node verify_artifacts.mjs`
H	`node reproduce.mjs`
I	`node package_submission.mjs`

Only E requests a paid API review; it requires `OPENAI_API_KEY` in the caller's environment. A can request a live review explicitly with `--live`; otherwise it replays saved evidence. Never paste a key into a document, chat, HTML, source code or submitted file.

Outer delivery items

File	What it is
`submission/The_Two_Calendars_Submission.zip`	The submission package
`submission/SHA256.txt`	Package checksum
`submission/archive_verification.json`	Package integrity checks
`submission/independent_archive_check.json`	Independent .NET inspection of the prior verified 54-file archive snapshot. It does not certify the bytes of an archive rebuilt after this document was added.

The package is rebuilt with command I after this document is added; the checksum and archive verification are regenerated from the rebuilt bytes.